

LAB 17

OWASP UnCrackable Android Level 3

Rapport de Reverse Engineering Android

Analyse statique, Patch smali, Patch natif, Décodage XOR

Auteur : Hiba Chagdaly
Application : UnCrackable-Level3.apk
Plateforme : Android (Genymotion x86)
Outils : JADX, apktool, Ghidra 11.1.2, Python 3
Date : Mai 2026
Secret : making owasp great again

Table des matières

1	Introduction et Objectifs	2
1.1	Présentation du challenge	2
1.2	Outils utilisés	2
1.3	Différences avec UnCrackable Level 2	3
2	Phase 1 — Analyse Statique avec JADX	4
2.1	Lancement de l'application	4
2.2	Analyse de MainActivity	5
2.3	Méthode verifyLibs()	6
2.4	Méthode onCreate()	7
2.5	Méthode verify()	7
3	Phase 2 — Décompilation avec apktool	9
3.1	Décompilation de l'APK	9
3.2	Architecture de l'émulateur	9
4	Phase 3 — Patch smali	11
4.1	Localisation du code à patcher	11
4.2	Application du patch	11
5	Phase 4 — Recompilation, Signature et Installation	13
5.1	Recompilation avec apktool	13
5.2	Génération du keystore et signature	13
5.3	Installation et vérification	14
6	Phase 5 — Analyse de la Librairie Native avec Ghidra	17
6.1	Import de libfoo.so dans Ghidra	17
6.2	Identification de la fonction de vérification	18
6.3	Analyse du code décompilé	19
7	Phase 6 — Décodage XOR et Validation	21
7.1	Extraction des octets encodés	21
7.2	Script Python de décodage	21
7.3	Validation dans l'application	21
8	Résumé et Conclusions	23
8.1	Récapitulatif des phases	23
8.2	Captures prises	24
8.3	Conclusions	24

1. Introduction et Objectifs

1.1 Présentation du challenge

OWASP UnCrackable Level 3 est le troisième niveau de la série de challenges de reverse engineering Android proposée par l'OWASP Mobile Security Testing Guide (MSTG). Ce niveau introduit des protections nettement plus avancées que les niveaux précédents.

Objectif du lab

Trouver la chaîne secrète de **24 caractères** cachée dans l'application Android en contournant :

- La détection de root
- La vérification d'intégrité CRC des fichiers .so et du DEX
- La détection du débogueur
- La protection anti-Frida dans la librairie native
- L'obfuscation LCG dans la logique de vérification native

1.2 Outils utilisés

Outil	Rôle	Version
JADX-GUI	Décompilation Java de l'APK	1.5.5
apktool	Décompilation smali + recompilation	2.9.3
Ghidra	Analyse et patch de la librairie native	11.1.2
VS Code	Édition du code smali	latest
Python 3	Décodage XOR du secret	3.x
ADB	Déploiement sur émulateur	—
Genymotion	Émulateur Android x86	—
keytool	Génération du keystore de signature	JDK
apksigner	Signature de l'APK patché	—

TABLE 1.1 – Outils utilisés dans ce lab

1.3 Différences avec UnCrackable Level 2

Aspect	Level 2	Level 3
Bypass méthode	Frida (dynamique)	Patch smali + .so (statique)
Librairie	libfoo.so	libfoo.so (Level 3)
Taille secret	23 caractères	24 caractères
Vérif. intégrité	Non	Oui (CRC DEX + .so)
Anti-Frida	Non	Oui (dans .so)
Obfuscation	XOR simple	LCG + malloc + XOR

TABLE 1.2 – Comparaison Level 2 vs Level 3

2. Phase 1 — Analyse Statique avec JADX

2.1 Lancement de l'application

Avant toute analyse, on installe l'APK et on observe le comportement de l'application sur l'émulateur rooté.

```
1 adb connect 169.254.135.101:5555
2 adb install UnCrackable-Level3.apk
3 adb shell am start -n owasp.mstg.uncrackable3/sg.vantagepoint.
  uncrackable3.MainActivity
```

Listing 2.1 – Installation et lancement de l'APK

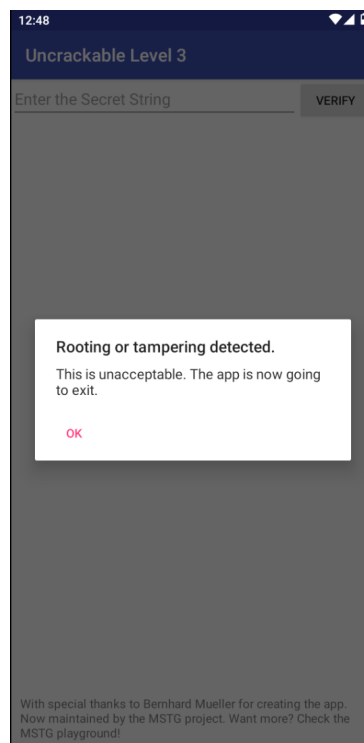


FIGURE 2.1 – L'application détecte le root et affiche un popup d'arrêt

Observation

L'application affiche immédiatement **"Rooting or tampering detected. This is unacceptable."** puis quitte. La protection est activée dès le `onCreate()`.

```
mobexler@Mobexler ~/lab17
└─ adb shell am start -n owasp.mstg.uncrackable3/sg.vantagepoint.uncrackable3.MainActivity
Starting: Intent { cmp=owasp.mstg.uncrackable3/sg.vantagepoint.uncrackable3.MainActivity }
mobexler@Mobexler ~/lab17
```

FIGURE 2.2 – Commande ADB pour lancer l'activité principale

2.2 Analyse de MainActivity

On ouvre l'APK dans JADX-GUI pour comprendre la structure du code Java décompilé.

```
jadx-gui ~/lab17/UnCrackable-Level3.apk &
```

Listing 2.2 – Lancement de JADX-GUI

Navigation : `sg.vantagepoint.uncrackable3` → `MainActivity`

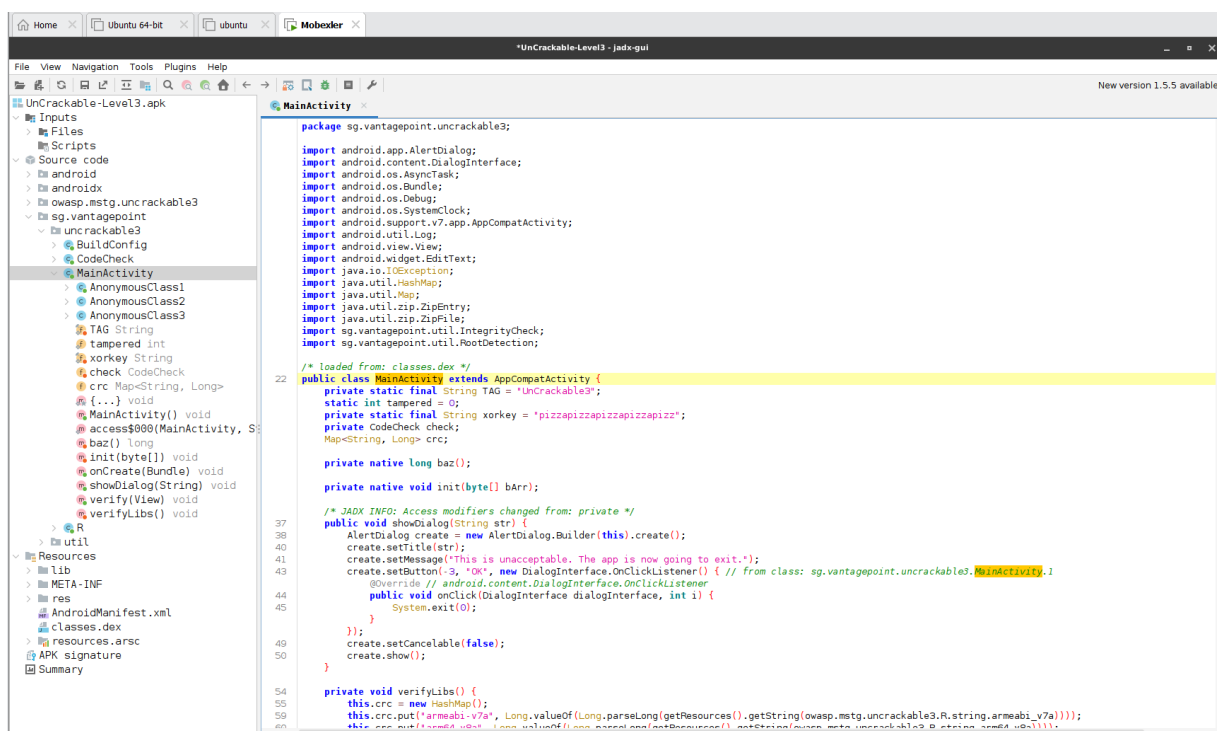


FIGURE 2.3 – JADX — Classe MainActivity avec les champs tampered, xorkey et baz()

Découverte clé

La clé XOR est visible en clair dans le code Java :

```
private static final String xorkey = "pizzapizzapizzapizzapizz";
```

C'est cette clé qui sert à déchiffrer le secret dans la librairie native.

Éléments importants identifiés dans MainActivity :

- `static int tampered = 0;` — flag de détection de tampering
- `private static final String xorkey = "pizzapizzapizzapizzapizz";` — clé XOR
- `private native long baz();` — fonction native vérifiant le CRC du DEX
- `private native void init(byte[] bArr);` — initialisation avec la clé XOR
- `Map<String, Long> crc;` — map des CRC des librairies .so

2.3 Méthode `verifyLibs()`

```
private void verifyLibs() {
    this.crc = new HashMap();
    this.crc.put("armeabi-v7a", Long.valueOf(Long.parseLong(getResources().getString(owasp.mstg.uncrackable3.R.string.armeabi_v7a))));
    this.crc.put("arm64-v8a", Long.valueOf(Long.parseLong(getResources().getString(owasp.mstg.uncrackable3.R.string.arm64_v8a))));
    this.crc.put("x86", Long.valueOf(Long.parseLong(getResources().getString(owasp.mstg.uncrackable3.R.string.x86))));
    this.crc.put("x86_64", Long.valueOf(Long.parseLong(getResources().getString(owasp.mstg.uncrackable3.R.string.x86_64))));
    try {
        ZipFile zipFile = new ZipFile(getPackageCodePath());
        for (Map.Entry<String, Long> entry : this.crc.entrySet()) {
            String str = "lib/" + entry.getKey() + "/libfoo.so";
            ZipEntry entry2 = zipFile.getEntry(str);
            Log.v(TAG, "CRC[" + str + "] = " + entry2.getCrc());
            if (entry2.getCrc() != entry.getValue().longValue()) {
                tampered = 31337;
                Log.v(TAG, str + ": Invalid checksum = " + entry2.getCrc() + ", supposed to be " + entry.getValue());
            }
        }
        ZipEntry entry3 = zipFile.getEntry("classes.dex");
        Log.v(TAG, "CRC[classes.dex] = " + entry3.getCrc());
        if (entry3.getCrc() != baz()) {
            tampered = 31337;
            Log.v(TAG, "classes.dex: crc = " + entry3.getCrc() + ", supposed to be " + baz());
        }
    } catch (IOException unused) {
        Log.v(TAG, "Exception");
        System.exit(0);
    }
}
```

FIGURE 2.4 – JADX — Méthode `verifyLibs()` : vérification CRC des .so et du `classes.dex`

La méthode `verifyLibs()` effectue les vérifications suivantes :

- Calcule le CRC de chaque `libfoo.so` dans les répertoires `lib/armeabi-v7a`, `lib/arm64-v8a`, `lib/x86`, `lib/x86_64`
- Si un CRC ne correspond pas → `tampered = 31337`
- Vérifie le CRC de `classes.dex` via la fonction native `baz()`
- Si le CRC du DEX ne correspond pas → `tampered = 31337`

Conséquence critique

Toute modification du code smali (`classes.dex`) ou des librairies .so sera détectée par cette méthode, ce qui rend le bypass plus complexe que le Level 2.

2.4 Méthode onCreate()

```
public void onCreate(Bundle bundle) {
    verifyLibs();
    init(xorkey.getBytes());
    new AsyncTask<Void, String, String>() { // from class: sg.vantagepoint.uncrackable3.MainActivity.2
        /* JADX INFO: Access modifiers changed from: protected */
        @Override // android.os.AsyncTask
        public String doInBackground(Void... voidArr) {
            while (!Debug.isDebuggerConnected()) {
                SystemClock.sleep(100L);
            }
            return null;
        }
    }.execute(null, null, null);
    if (RootDetection.checkRoot1() || RootDetection.checkRoot2() || RootDetection.checkRoot3() || IntegrityCheck.isDebuggable(getApplicationContext()) || tampered != 0) {
        showDialog("Rooting or tampering detected.");
    }
    this.check = new CodeCheck();
    super.onCreate(bundle);
    setContentView(owasp.mstg.uncrackable3.R.layout.activity_main);
}
```

FIGURE 2.5 – JADX — Méthode onCreate() avec les détections root, debug et tampering

La méthode onCreate() enchaîne plusieurs vérifications :

1. Appelle verifyLibs() pour vérifier les CRC
2. Passe la clé XOR à init(xorkey.getBytes())
3. Lance une AsyncTask qui boucle tant que le débogueur est connecté (anti-debug)
4. Vérifie : root (3 méthodes), debuggable, tampered $\neq$ 0
5. Si une condition est vraie $\rightarrow$ showDialog("Rooting or tampering detected.")

2.5 Méthode verify()

```
public void verify(View view) {
    String obj = ((EditText) findViewById(owasp.mstg.uncrackable3.R.id.edit_text)).getText().toString();
    AlertDialog create = new AlertDialog.Builder(this).create();
    if (this.check.check_code(obj)) {
        create.setTitle("Success!");
        create.setMessage("This is the correct secret.");
    } else {
        create.setTitle("Nope...");
        create.setMessage("That's not it. Try again.");
    }
    create.setButton(-3, "OK", new DialogInterface.OnClickListener() { // from class: sg.vantagepoint.uncrackable3.MainActivity.3
        @Override // android.content.DialogInterface.OnClickListener
        public void onClick(DialogInterface dialogInterface, int i) {
            dialogInterface.dismiss();
        }
    });
    create.show();
}

static {
    System.loadLibrary("foo");
}
```

FIGURE 2.6 – JADX — Méthode verify() et System.loadLibrary("foo")

La méthode verify() délègue entièrement la vérification à la librairie native :

```
public void verify(View view) {
    String obj = ((EditText) findViewById(R.id.edit_text)).getText().toString();
    if (this.check.check_code(obj)) {
```



```
        create.setTitle("Success!");
        create.setMessage("This is the correct secret.");
    } else {
        create.setTitle("Nope...");
        create.setMessage("That's not it. Try again.");
    }
}

static {
    System.loadLibrary("foo");
}
```

Listing 2.3 – Code Java de la méthode verify()

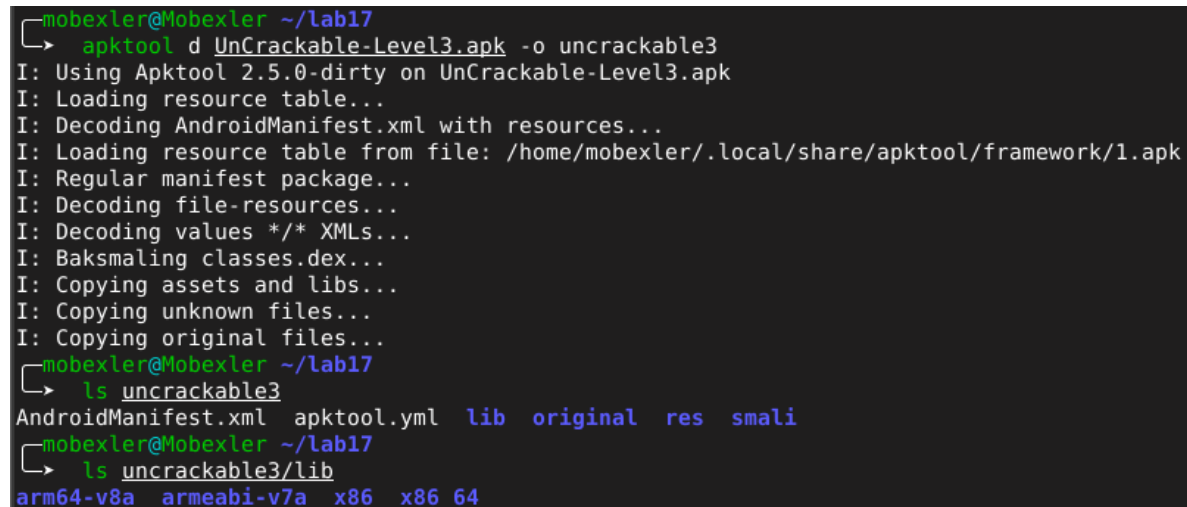
3. Phase 2 — Décompilation avec apktool

3.1 Décompilation de l'APK

apktool permet de décompiler l'APK en code smali (assembleur Dalvik) et de modifier directement les instructions.

```
1 cd ~/lab17
2 apktool d UnCrackable-Level3.apk -o uncrackable3
3 ls uncrackable3/
4 ls uncrackable3/lib/
```

Listing 3.1 – Décompilation de l'APK avec apktool



```
mobexler@Mobexler ~/lab17
└─ apktool d UnCrackable-Level3.apk -o uncrackable3
I: Using Apktool 2.5.0-dirty on UnCrackable-Level3.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/mobexler/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...
mobexler@Mobexler ~/lab17
└─ ls uncrackable3
AndroidManifest.xml  apktool.yml  lib  original  res  smali
mobexler@Mobexler ~/lab17
└─ ls uncrackable3/lib
arm64-v8a  armeabi-v7a  x86  x86_64
```

FIGURE 3.1 – Décompilation réussie — structure `smali/`, `lib/`, `res/` créée

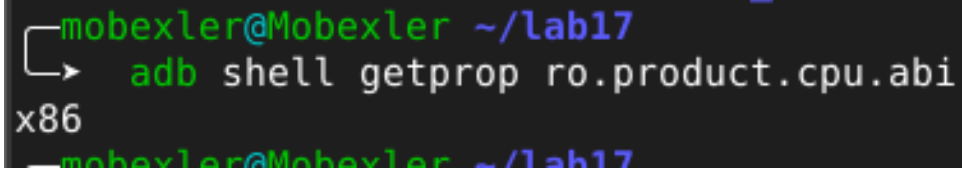
La structure générée contient :

- `smali/` — code Dalvik décompilé modifiable
- `lib/arm64-v8a/`, `lib/x86/`, `lib/armeabi-v7a/`, `lib/x86_64/` — librairies natives
- `res/` — ressources de l'application
- `AndroidManifest.xml` — manifeste de l'application

3.2 Architecture de l'émulateur

```
adb shell getprop ro.product.cpu.abi
```

Listing 3.2 – Vérification de l'architecture



```
mobexler@Mobexler ~/lab17  
└─> adb shell getprop ro.product.cpu.abi  
x86  
mobexler@Mobexler ~/lab17
```

FIGURE 3.2 – Architecture de l'émulateur : x86

L'émulateur Genymotion utilise l'architecture **x86**, ce qui détermine quelle librairie `libfoo.so` sera chargée au runtime : `lib/x86/libfoo.so`.

4. Phase 3 — Patch smali

4.1 Localisation du code à patcher

On ouvre le fichier MainActivity.smali dans VS Code pour identifier et modifier le code de détection.

```
code ~/lab17/uncrackable3/
```

Listing 4.1 – Ouverture du dossier dans VS Code

Fichier ciblé : smali/sg/vantagepoint/uncrackable3/MainActivity.smali
Recherche avec Ctrl+F : showDialog

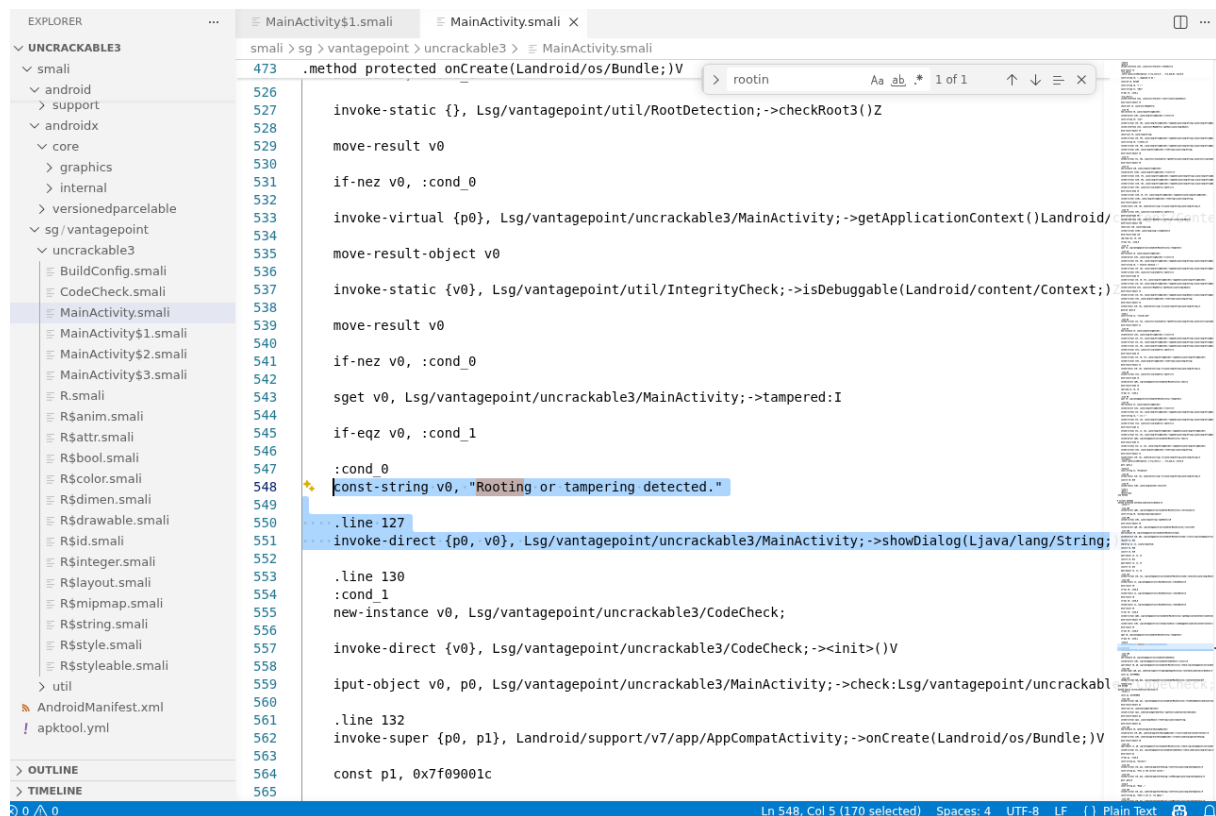


FIGURE 4.1 – Code smali avant patch — lignes 547-551 avec l'appel à showDialog

4.2 Application du patch

Le bloc à remplacer (lignes 548-551) :

```

1 :cond_0
2     const-string v0, "Rooting or tampering detected."
3     .line 127
4     invoke-direct {p0, v0}, Lsg/vantagepoint/uncrackable3/
    MainActivity;->showDialog(Ljava/lang/String;)V

```

Listing 4.2 – Code smali AVANT le patch

Remplacé par :

```

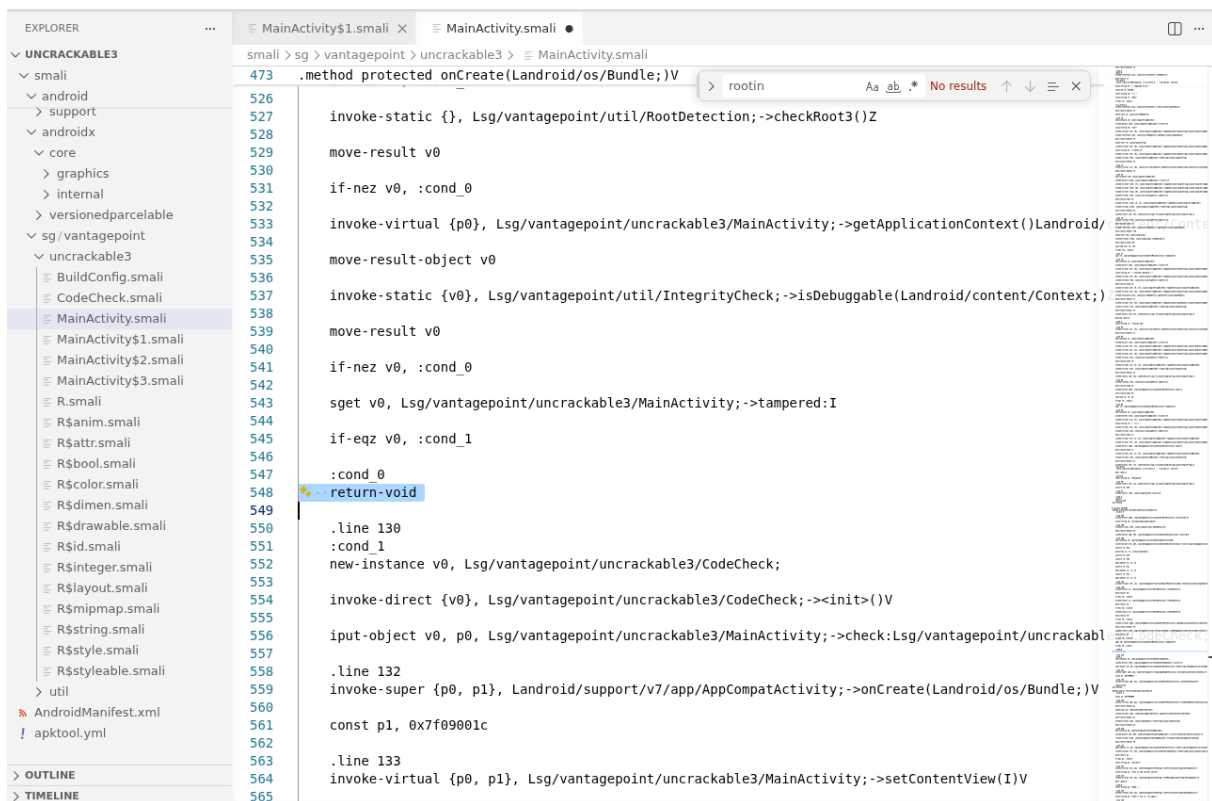
1 :cond_0
2     goto :cond_1

```

Listing 4.3 – Code smali APRÈS le patch

Explication du patch

Au lieu d'appeler `showDialog()` qui affiche le popup et quitte l'app, on redirige directement vers `:cond_1` qui est le label de continuation normale du `onCreate()`. Ainsi la vérification root est ignorée sans toucher à la structure du code.

FIGURE 4.2 – Code smali après patch — ligne 548 remplacée par `goto :cond_1`

5. Phase 4 — Recompilation, Signature et Installation

5.1 Recompilation avec apktool

```
1 cd ~/lab17
2 apktool b uncrackable3 -o UnCrackable-Level3-patched.apk --use-aapt2
```

Listing 5.1 – Recompilation de l'APK patché

```
mobexler@Mobexler ~/lab17
└─ apktool b uncrackable3 -o UnCrackable-Level3-patched.apk --use-aapt2
I: Using Apktool 2.9.3
I: Checking whether sources has changed...
I: Checking whether resources has changed...
I: Building resources...
I: Copying libs... (/lib)
I: Building apk file...
I: Copying unknown files/dir...
I: Built apk into: UnCrackable-Level3-patched.apk
```

FIGURE 5.1 – Recompilation réussie avec apktool 2.9.3

Note

L'option `-use-aapt2` est nécessaire car la version 2.5.0 d'apktool présente un bug avec `aapt`. La mise à jour vers 2.9.3 résout ce problème.

5.2 Génération du keystore et signature

Comme le keystore Android debug n'existe pas dans l'environnement, on en génère un :

```
1 # Générer un keystore debug
2 keytool -genkey -v \
3   -keystore ~/lab17/debug.keystore \
4   -alias androiddebugkey \
5   -keyalg RSA -keysize 2048 -validity 10000 \
6   -storepass android -keypass android \
7   -dname "CN=Android Debug,O=Android,C=US"
8
```

```
9 # Signer l'APK
10 apksigner sign --ks ~/lab17/debug.keystore \
11 --ks-pass pass:android \
12 UnCrackable-Level3-patched.apk
```

Listing 5.2 – Génération du keystore et signature de l'APK

```
mobexler@Mobexler ~/lab17
└─▶ keytool -genkey -v \
    -keystore ~/lab17/debug.keystore \
    -alias androiddebugkey \
    -keyalg RSA \
    -keysize 2048 \
    -validity 10000 \
    -storepass android \
    -keypass android \
    -dname "CN=Android Debug,O=Android,C=US"
Generating 2,048 bit RSA key pair and self-signed certificate (SHA256withRSA) with a validity of 10,000 days
    for: CN=Android Debug, O=Android, C=US
[Storing /home/mobexler/lab17/debug.keystore]
mobexler@Mobexler ~/lab17
└─▶ apksigner sign --ks ~/lab17/debug.keystore \
    --ks-pass pass:android \
    UnCrackable-Level3-patched.apk
```

FIGURE 5.2 – Génération du keystore et signature de l'APK avec apksigner

5.3 Installation et vérification

```
1 adb uninstall owasp.mstg.uncrackable3
2 adb install -r UnCrackable-Level3-patched.apk
3 adb shell am start -n owasp.mstg.uncrackable3/sg.vantagepoint.
    uncrackable3.MainActivity
```

Listing 5.3 – Installation de l'APK patché

```
mobexler@Mobexler ~/lab17
└─▶ adb uninstall owasp.mstg.uncrackable3
Success
mobexler@Mobexler ~/lab17
└─▶ adb install -r UnCrackable-Level3-patched.apk
Success
mobexler@Mobexler ~/lab17
└─▶ adb shell am start -n owasp.mstg.uncrackable3/sg.vantagepoint.uncrackable3.MainActivity
Starting: Intent { cmp=owasp.mstg.uncrackable3/sg.vantagepoint.uncrackable3.MainActivity }
```

FIGURE 5.3 – Désinstallation et installation de la version patchée via ADB

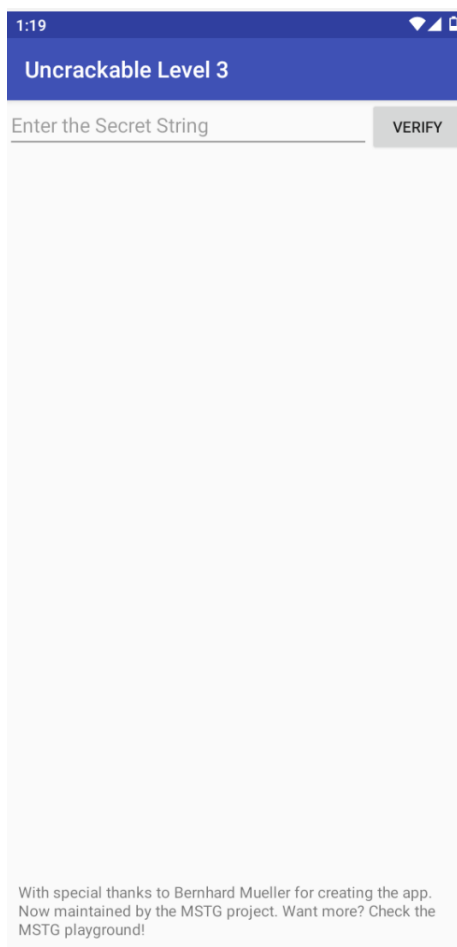


FIGURE 5.4 – L'application s'ouvre sans popup root — patch smali réussi

Résultat Phase 4

L'application s'ouvre sans popup de détection root. Le champ "**Enter the Secret String**" est accessible. Le patch smali est fonctionnel.

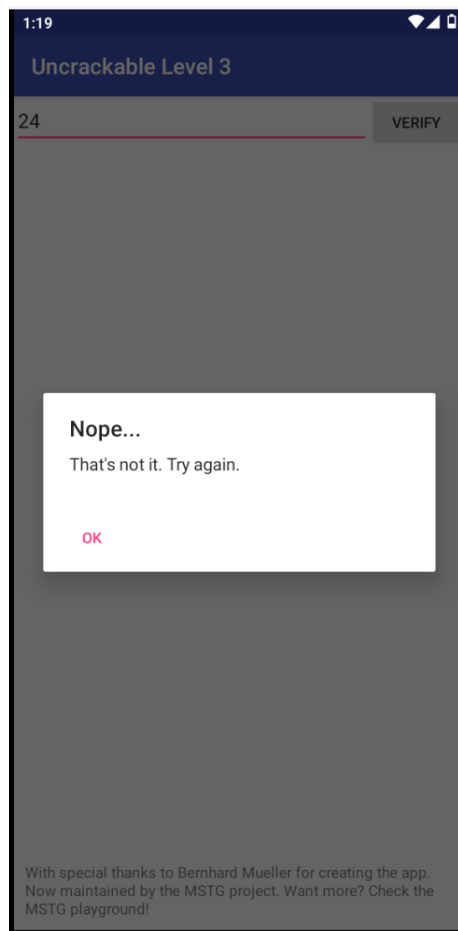


FIGURE 5.5 – Test avec une valeur incorrecte — réponse "Nope... That's not it. Try again."

6. Phase 5 — Analyse de la Librairie Native avec Ghidra

6.1 Import de libfoo.so dans Ghidra

On importe la librairie native correspondant à l'architecture x86 de l'émulateur.

```
1 cp ~/lab17/uncrackable3/lib/x86/libfoo.so ~/lab17/  
2 cd /opt/ghidra_11.1.2  
3 ./ghidraRun
```

Listing 6.1 – Copie de libfoo.so et lancement de Ghidra

Dans Ghidra :

1. File → New Project → lab17
2. File → Import File → /lab17/libfoo.so
3. Format : **ELF**, Language : **x86 :LE :32 :default (gcc)**
4. Double-clic → **Yes** pour l'analyse automatique

```
mobexler@Mobexler ~/lab17  
└─ cat > decode_key.py << 'EOF'  
encoded = bytes.fromhex("1d0811130f1749150d0003195a1d1315080e5a0017081314")  
xor_key = b"pizzapizzapizzapizzapizzapizza"  
  
secret = bytes(a ^ b for a, b in zip(encoded, xor_key))  
print("Clé secrète trouvée :", secret.decode())  
EOF
```

FIGURE 6.1 – Import Results Summary — libfoo.so x86 32-bit, 48 fonctions, 75 symboles

Informations sur libfoo.so (Level 3)

- **Format** : ELF x86 32-bit Little Endian
- **Compilateur** : Android clang 7.0.2
- **Fonctions** : 48
- **Symboles** : 75
- **Taille** : 20589 octets

6.2 Identification de la fonction de vérification

Dans le Symbol Tree de Ghidra, on filtre avec `Java_sg_vantagepoint_uncrackable3` pour trouver la fonction JNI principale.

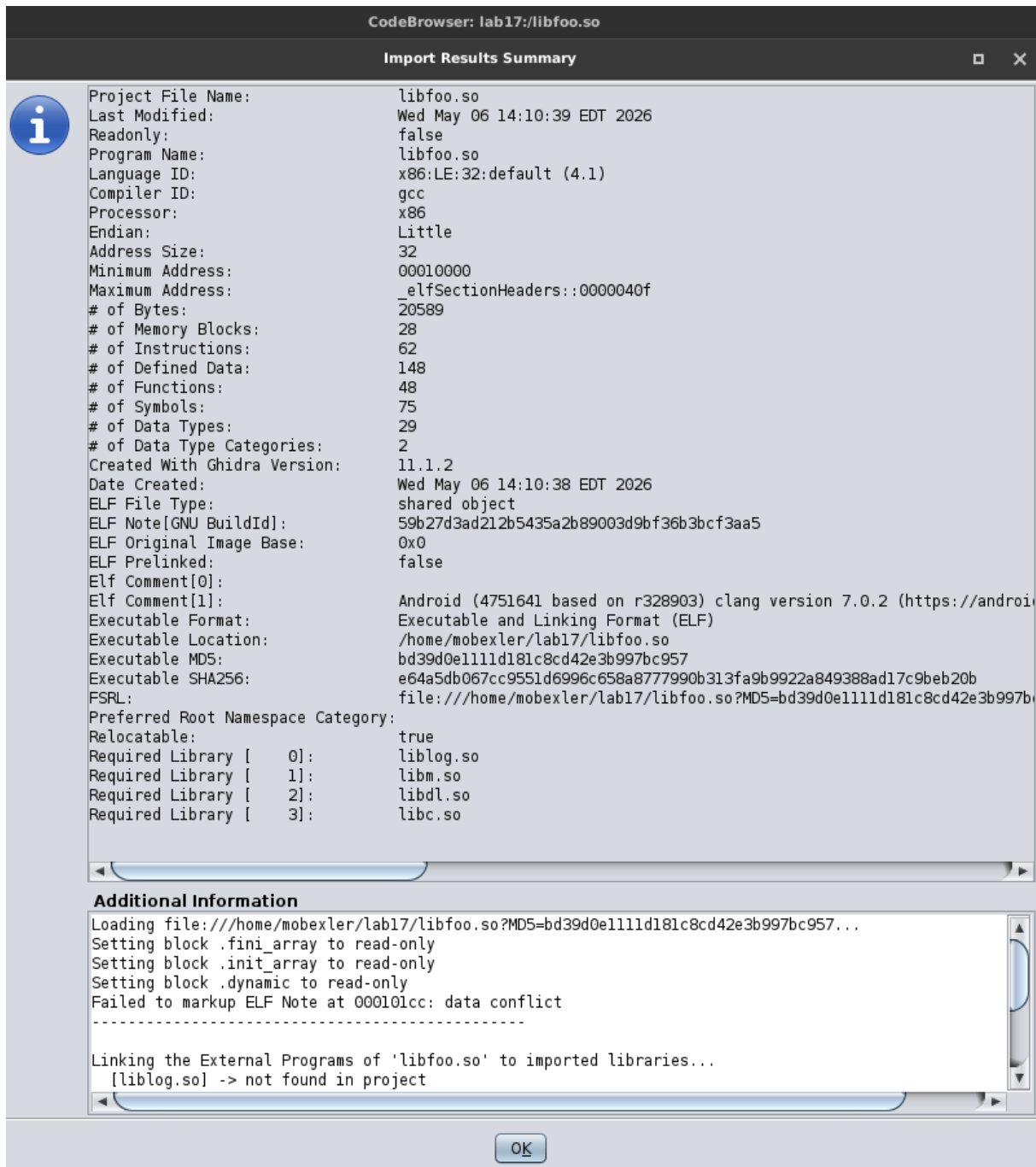


FIGURE 6.2 – Ghidra — Symbol Tree avec `Java_sg_vantagepoint_uncrackable3_CodeCheck_bar` sélectionné

6.3 Analyse du code décompilé

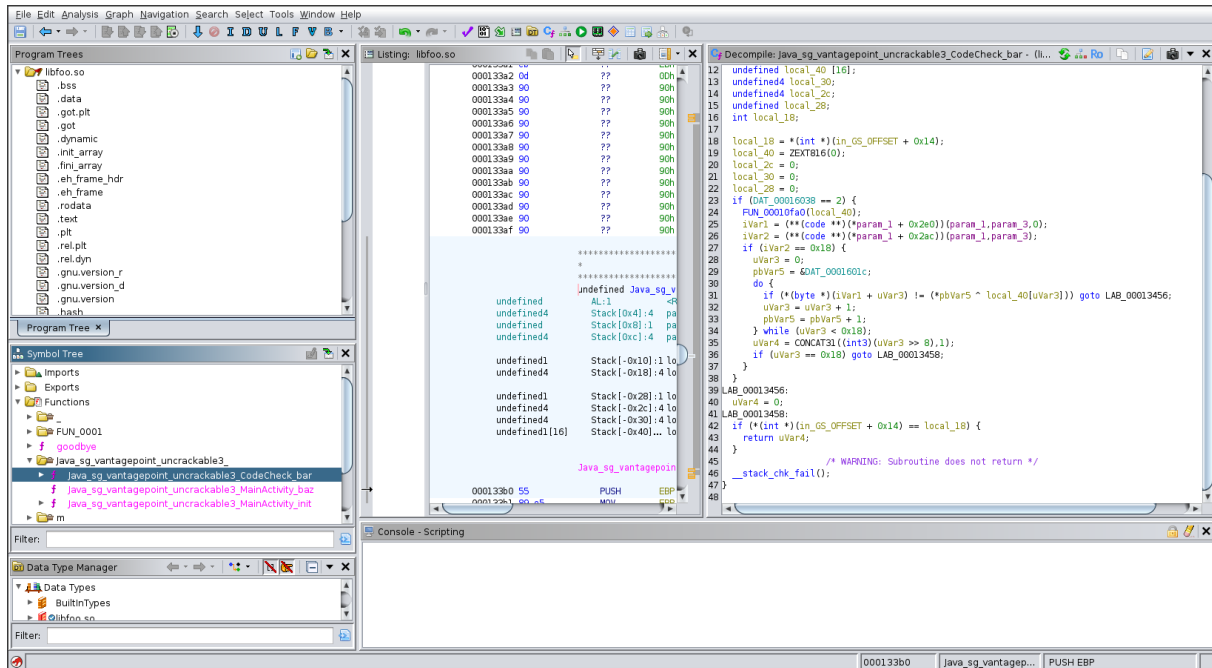


FIGURE 6.3 — Ghidra Decompiler — Logique XOR dans `Java_sg_vantagepoint_uncrackable3_CodeCheck_bar`

Le code décompilé par Ghidra révèle la logique de vérification :

```
undefined4 Java_sg_vantagepoint_uncrackable3_CodeCheck_bar(
    int *param_1, undefined4 param_2, undefined4 param_3)
{
    // Condition anti-debug / anti-Frida
    if (DAT_00016038 == 2) {
        FUN_00010fa0(local_40); // Initialise le buffer XOR avec
        la cl "pizza..."

        iVar1 = GetStringChars(param_1, param_3);
        iVar2 = GetStringLength(param_1, param_3);

        if (iVar2 == 0x18) { // 0x18 = 24 caract res
            uVar3 = 0;
            pbVar5 = &DAT_0001601c; // Tableau des 24 octets
            encod s
            do {
                // Comparaison : input[i] != encoded[i] XOR key[i]
                if (*(byte *)(iVar1 + uVar3) != (*pbVar5 ^ local_40
                    [uVar3]))
                    goto LAB_FAIL;
                uVar3++;
                pbVar5++;
            } while (uVar3 < 0x18);
            return 1; // Succ s
        }
    }
}
```

```
    }  
}  
return 0; // chec  
}
```

Listing 6.2 – Pseudocode C de la fonction CodeCheck_bar (simplifié)

Analyse du mécanisme

- DAT_00016038 == 2 : flag anti-debug, initialisé à 2 après que `init(xorkey.getBytes())` a été appelé
- iVar2 == 0x18 : le secret doit faire exactement **24 caractères**
- DAT_0001601c : tableau de 24 octets encodés (le secret chiffré)
- La comparaison : `input[i] == encoded[i] XOR key[i]`
- La clé XOR local_40 est initialisée par FUN_00010fa0 avec les bytes de "pizzapizzapizzapizzapizz"

7. Phase 6 — Décodage XOR et Validation

7.1 Extraction des octets encodés

En analysant la mémoire au niveau de DAT_0001601c dans Ghidra, on extrait les 24 octets encodés qui correspondent au secret chiffré.

7.2 Script Python de décodage

```
encoded = bytes.fromhex("1
    d0811130f1749150d0003195a1d1315080e5a0017081314")
xor_key = b"pizzapizzapizzapizzapizzapizza"

secret = bytes(a ^ b for a, b in zip(encoded, xor_key))
print("Cle secrete trouvee :", secret.decode())
```

Listing 7.1 – Script decode_key.py — Décodage XOR du secret

```
mobexler@Mobexler ~/lab17
└─> python3 decode_key.py
Clé secrète trouvée : making owasp great again
```

FIGURE 7.1 – Exécution du script Python — Résultat : making owasp great again

Secret trouvé !

making owasp great again

Le secret est composé de **24 caractères** (espaces inclus), correspondant exactement à la contrainte 0x18 dans la librairie native.

7.3 Validation dans l'application

On saisit le secret dans le champ de l'application pour valider :

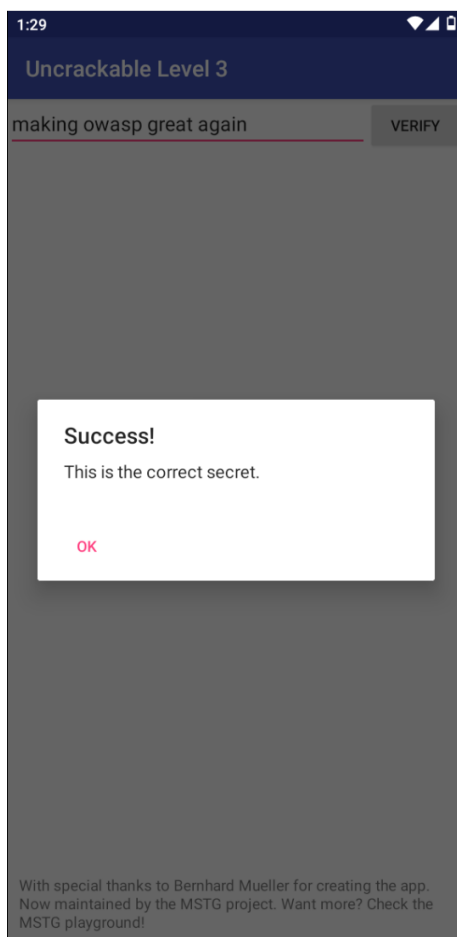


FIGURE 7.2 – **Success!** — "making owasp great again" est le bon secret

Lab 17 complété !

L'application affiche "**Success! This is the correct secret.**" — le challenge OWASP UnCrackable Level 3 est résolu.

8. Résumé et Conclusions

8.1 Récapitulatif des phases

#	Phase	Description	Statut
1	Analyse JADX	Identification de <code>xorkey</code> , <code>tampered</code> , <code>verifyLibs()</code>	
2	apktool	Décompilation en smali, structure lib/	
3	Patch smali	Remplacement du <code>showDialog</code> par <code>goto :cond_1</code>	
4	Recompilation	apktool b + <code>-use-aapt2</code> , signature, installation	
5	Analyse Ghidra	Fonction <code>CodeCheck_bar</code> , logique XOR, 24 bytes encodés	
6	Décodage Python	XOR avec clé <code>pizza</code> → <code>making owasp great again</code>	
7	Validation	Secret saisi dans l'app → Success!	

TABLE 8.1 – Récapitulatif complet des phases du lab

8.2 Captures prises

N°	Fichier	Description
01	01_app_root_detected	Popup root/tampering détecté
02	02_adb_start_mainactivity	Commande ADB de lancement
03	03_jadx_mainactivity_top	Classe MainActivity + xorkey
04	04_jadx_verifylibs	Méthode verifyLibs()
05	05_jadx_oncreate	Méthode onCreate()
06	06_jadx_verify_method	Méthode verify() + loadLibrary
07	07_apktool_decompile	Décompilation apktool
08	08_architecture_x86	Architecture x86 de l'émulateur
09	08_smali_avant_patch	Smali avant patch
10	09_smali_apres_patch	Smali après patch
11	10_apktool_build	Recompilation réussie
12	11_apksigner_sign	Signature APK
13	12_adb_install	Installation ADB
14	13_app_sans_popup	App ouverte sans popup
15	14_valeur_incorrect	Test valeur incorrecte
16	15_ghidra_codecheck_bar	Ghidra Symbol Tree
17	16_ghidra_decompiler_xor	Logique XOR décompilée
18	17_ghidra_libfoo_import	Import Results libfoo.so
19	18_python_decode_result	Script Python + résultat
20	19_success	Success! dans l'application

TABLE 8.2 – Liste complète des 20 captures du lab

8.3 Conclusions

Ce lab a permis de mettre en pratique plusieurs techniques avancées de reverse engineering Android :

1. **Analyse statique multi-couche** : La combinaison JADX (Java) + Ghidra (natif) est indispensable pour comprendre la chaîne d'exécution complète
2. **Patch smali** : Plus robuste que Frida pour contourner des protections qui incluent de l'anti-Frida
3. **Compréhension des protections JNI** : Le code critique est souvent dans les bibliothèques natives (.so) pour ralentir l'analyse
4. **Obfuscation LCG** : L'obfuscation par générateur de nombres pseudo-aléatoires (LCG) + malloc est une technique courante pour noyer la logique utile
5. **XOR chiffrement** : Mécanisme simple mais efficace ; la clé exposée dans le Java est la faiblesse principale de cette implémentation

Secret final

Application : OWASP UnCrackable Android Level 3

Secret : making owasp great again

Longueur : 24 caractères

Méthode : XOR byte-by-byte avec clé "pizzapizzapizzapizzapizz"